

1.3.2.1 Abusing Fragmentation & Detection

Let's talk briefly about fragmentation. Fragmentation is the action of dividing a packet whose size is greater than the Maximum Transmission Unit (MTU) into equal-sized (except for the last) packets, whose size is less or equal to the MTU. Fragmentation can be performed by a router or the sending host itself.

Each fragment's IP header contains fields and values that facilitate reassembling the original packet at the destination.

1.3.2.1 Abusing Fragmentation & Detection

What is of interested to us, is that fragmentation is oftentimes abused for IDS/IPS evasion purposes.

When it comes to fragmented packets, IDS/IPS must act just if they were the destination host, in terms of packet reassembling. This is for obvious reasons, IDS/IPS need the whole packet in order to inspect it.

1.3.2.1 Abusing Fragmentation & Detection

That being said, attackers can introduce difficulties in the reassembling procedure by the IDS/IPS, such as:

- Crafted fragmented packets with identical offsets but different payloads
- Crafted packets arriving with a great time difference

```
21 def __init__(self, control = 0, control = 0)
22     self.control = control
23     self.observations = {}
24     self.candidates = {}
25     self.evaluate_candidates()
26
27 def initialize(experiment, observations = {}, control = 0)
28     self.experiment = experiment
29     self.observations = observations
30     self.candidates = {}
31     self.evaluate_candidates()
32
33 def freeze()
34     pass
35
36 # TODO: the experiment's control
37 def control()
38     return self.experiment.control
39
40 # TODO: the name of the experiment
41 def experiment_name()
42     return self.experiment.name
43
44 # TODO: the result of the experiment
45 def result()
46     return self.experiment.result
47
48 # TODO: the result of the experiment
49 def result()
50     return self.experiment.result
51
52 # TODO: the result of the experiment
53 def result()
54     return self.experiment.result
```



1.3.2.1 Abusing Fragmentation & Detection

For the IDS/IPS to safely perform such packet reassembling and inspection, it should act just like the destination host does.

Let's consider the delayed fragments case. If the IDS/IPS, due to performance limitations, doesn't wait as long as the destination does for a fragment to arrive, a delayed fragment containing a malicious payload could evade it and exploit the destination.

1.3.2.1 Abusing Fragmentation & Detection

Consider the *frag-based_scan.pcap* file found in this module's resources. Do you think this traffic is malicious?

```
>> tcpdump -r frag-based_scan.pcap -ntx -tttt -v
```

```
# tcpdump -r frag-based_scan.pcap -ntx -tttt -v
reading from file frag-based_scan.pcap, link-type EN10MB (Ethernet)
00:00:00.000000 IP (tos 0x0, ttl 64, id 47486, offset 0, flags [+], proto ICMP (1), length 28)
  192.168.1.6 > 192.168.1.4: ICMP echo request, id 33286, seq 0, length 8
    0x0000: 4500 001c b97e 2000 4001 1e08 c0a8 0106
    0x0010: c0a8 0104 0800 75f9 8206 0000 0000 0000
    0x0020: 0000 0000 0000 0000 0000 0000 0000 0000
00:00:31.881589 IP (tos 0xc0, ttl 64, id 39831, offset 0, flags [none], proto ICMP (1), length 56)
  192.168.1.4 > 192.168.1.6: ICMP ip reassembly time exceeded, length 36
    IP (tos 0x0, ttl 64, id 47486, offset 0, flags [+], proto ICMP (1), length 28)
  192.168.1.6 > 192.168.1.4: ICMP echo request, id 33286, seq 0, length 8
    0x0000: 45c0 0038 9b97 0000 4001 5b13 c0a8 0104
    0x0010: c0a8 0106 0b01 f4fe 0000 0000 4500 001c
```

1.3.2.1 Abusing Fragmentation & Detection

Consider the *frag-based_scan.pcap* file found in this module's resources. Do you think this traffic is malicious?

```
>> tcpdump -r frag-based_scan.pcap -ntx -tttt -v
```

Fragment ID

Fragment offset

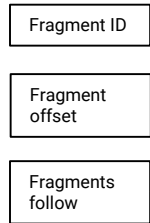
Fragments follow

```
# tcpdump -r frag-based_scan.pcap -ntx -tttt -v
reading from file frag-based_scan.pcap, link-type EN10MB (Ethernet)
00:00:00.000000 IP (tos 0x0, ttl 64, id 47486, offset 0, flags [+], proto ICMP (1), length 28)
192.168.1.6 > 192.168.1.4: ICMP echo request, id 33286, seq 0, length 8
0x0000: 4500 001c b97e 2000 4001 1e08 c0a8 0106
0x0010: c0a8 0104 0800 75f9 8206 0000 0000 0000
0x0020: 0000 0000 0000 0000 0000 0000 0000
00:00:31.881589 IP (tos 0xc0, ttl 64, id 39831, offset 0, flags [none], proto ICMP (1), length 56)
192.168.1.4 > 192.168.1.6: ICMP ip reassembly time exceeded, length 36
IP (tos 0x0, ttl 64, id 47486, offset 0, flags [+], proto ICMP (1), length 28)
192.168.1.6 > 192.168.1.4: ICMP echo request, id 33286, seq 0, length 8
0x0000: 45c0 0038 9b97 0000 4001 5b13 c0a8 0104
0x0010: c0a8 0106 0b01 f4fe 0000 0000 4500 001c
```

1.3.2.1 Abusing Fragmentation & Detection

Consider the *frag-based_scan.pcap* file found in this module's resources. Do you think this traffic is malicious?

```
>> tcpdump -r frag-based_scan.pcap -ntx -tttt -v
```



```
# tcpdump -r frag-based_scan.pcap -ntx -tttt -v
reading from file frag-based_scan.pcap, link-type EN10MB (Ethernet)
00:00:00.000000 IP (tos 0x0, ttl 64, id 47486, offset 0, flags [+], proto ICMP (1), length 28)
192.168.1.6 > 192.168.1.4: ICMP echo request, id 33286, seq 0, length 8
0x0000: 4500 001c b97e 2000 4001 1e08 c0a8 0106
0x0010: c0a8 0104 0800 75f9 8206 0000 0000 0000
0x0020: 0000 0000 0000 0000 0000 0000 0000 0000
00:00:31.881589 IP (tos 0xc0, ttl 64, id 39831, offset 0, flags [none], proto ICMP (1), length 56)
192.168.1.4 > 192.168.1.6: ICMP ip reassembly time exceeded, length 36
IP (tos 0x0, ttl 64, id 47486, offset 0, flags [+], proto ICMP (1), length 28)
192.168.1.6 > 192.168.1.4: ICMP echo request, id 33286, seq 0, length 8
0x0000: 45c0 0038 9b97 0000 4001 5b13 c0a8 0104
0x0010: c0a8 0106 0b01 f4fe 0000 0000 4500 001c
```

HINT: A stealthy mapping technique exists that leverages fragmentation. Specifically, the attacker sends an incomplete set of fragments (the zero offset fragment must be included) to the scanned hosts. Of course, the scanned hosts should be listening on the specified port or protocol.

If this is the case, once the first fragment is received, a timer is set. Once this timer expires, the receiving host transmits an ICMP "Fragment reassembly time exceeded" error back to the attacker.

1.3.2.1 Abusing Fragmentation & Detection

Consider the *frag-based_scan.pcap* file found in this module's resources. Do you think this traffic is malicious?

```
>> tcpdump -r frag-based_scan.pcap -ntx -tttt -v
```

1. Although there is a flag designating that fragments will follow, we see no fragments arriving for the next 30 seconds

2. Ultimately, we notice an ICMP "reassembly time exceeded" error being sent back to the source that transmitted the incomplete fragment.

We are either dealing with a host misconfiguration or a stealthy scanning attempt.

```
# tcpdump -r frag-based_scan.pcap -ntx -tttt -v
reading from file frag-based_scan.pcap, link-type EN10MB (Ethernet)
00:00:00.000000 IP (tos 0x0, ttl 64, id 47486, offset 0, flags [+], proto ICMP (1), length 28)
  192.168.1.6 > 192.168.1.4: ICMP echo request, id 33286, seq 0, length 8
    0x0000: 4500 001c b97e 2000 4001 1e08 c0a8 0106
    0x0010: c0a8 0104 0800 75f9 8206 0000 0000 0000
    0x0020: 0000 0000 0000 0000 0000 0000 0000 0000
00:00:31.881589 IP (tos 0xc0, ttl 64, id 39831, offset 0, flags [none], proto ICMP (1), length 56)
  192.168.1.4 > 192.168.1.6: ICMP ip reassembly time exceeded, length 36
    IP (tos 0x0, ttl 64, id 47486, offset 0, flags [+], proto ICMP (1), length 28)
  192.168.1.6 > 192.168.1.4: ICMP echo request, id 33286, seq 0, length 8
    0x0000: 45c0 0038 9b97 0000 4001 5b13 c0a8 0104
    0x0010: c0a8 0106 0b01 f4fe 0000 0000 4500 001c
```

1.3.2.1 Abusing Fragmentation & Detection

Let's see more malicious fragmentation examples.

An old DoS technique was sending an IP packet exceeding the 65535 bytes limit of data via a ping command. As you can imagine, this overly big packet would be fragmented and reassembled at the destination host. When older Operating Systems tried to reassemble such a packet they experienced system crashes, reboots or major degradations in performance.



1.3.2.1 Abusing Fragmentation & Detection

In a traffic capture of such an attack, you should see the following moments before the vulnerable system crashed.

This attack is known as ping-of-death.

```
0x05b0: 4141 4141 4141 4141 4141 4141 4141 4141
0x05c0: 4141 4141 4141 4141 4141 4141 4141 4141
0x05d0: 4141 4141 4141 4141 4141 4141
00:00:00.032431 IP (tos 0x0, ttl 64, id 21223, offset 65120, flags [+], proto ICMP (1), length 1500)
```

Right after that, the value of the reassembled packet will exceed the 65535 bytes limit.

1.3.2.1 Abusing Fragmentation & Detection

Another malicious fragmentation example is the old and UDP-based Teardrop attack.

In this case, crafted overlapping fragments were utilized in order to introduce ambiguities in the reassembling procedure and cause vulnerable systems to crash.

Teardrop attack on the wire

```
* Frame 8: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on
  Ethernet II, Src: AddronT_d9:7c:fd (00:a0:c3:d9:7c:fd), Dst: Toshiba_cf:d9:cd (00:00:39:cf:d9:cd)
  Internet Protocol Version 4, Src: 10.1.1.1, Dst: 129.111.30.27
    9100 ... = Version: 4
      * 9101 = Header Length: 28 bytes (5)
        Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
        Total length: 56
        Identification: 0x00f2 (242)
        Flags: 0x01 (More Fragments)
        Fragment offset: 0
        Time to live: 64
        Protocol: UDP (17)
        Header checksum: 0xaf37 [validation disabled]
        Header checksum status: Unverified
        Source: 10.1.1.1
        Destination: 129.111.30.27
        [Source GeoIP: Unknown]
        [Destination GeoIP: AS26971 University of Texas, United States, San Antonio, TX, 29.597208, -98.574799]
        Reassembled IPv4 in frame: 9
      * Data (36 bytes)
        Data: 7ab4ee50024000000000000000000000000000000000...
          [Length: 36]
```

```

9 39.016348      10.1.1.1          129.111.30.27    UDP      38 31915 - 20197 [BAD_UDP_LENGTH 36 > IP_PAYLOAD_LENGTH] Len=28
10 35.285494      Addrttont_d9:7c:fd Toshiba.cf:d9:c0 ARP      42 Who has 10.0.0.254? Tell 10.0.0.0
11 285487         Addrttont_d9:7c:fd Toshiba.cf:d9:c0 ARP      42 Who has 10.0.0.254? Tell 10.0.0.0
12 37.285485      Addrttont_d9:7c:fd Toshiba.cf:d9:c0 ARP      42 Who has 10.0.0.254? Tell 10.0.0.0
13 38.285590      Addrttont_d9:7c:fd Broadcast     ARP      42 Who has 10.0.0.254? Tell 10.0.0.0
14 38.287366      Addrttont_d9:7c:fd Addrttont_d9:7c:fd ARP      60 10.0.0.254 is at 00:00:39:cf:d9:c0
15 40.001851      Ciscoc_7c:eb:3d   Ciscoc_7c:eb:3d LOOP     60 Reply
16 47.973426      10.0.0.0          10.0.0.254      ICMP     98 Echo (ping) request id=0xc41b, seq=0/0, ttl=64 (reply in 17)
17 47.977697      10.0.0.254        10.0.0.0        ICMP     98 Echo (ping) reply id=0xc41b, seq=0/0, ttl=255 (request in 16)

[... Sniffed ...]

IPv4 Header Length: 20 bytes (6)
+ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 24
Identification: 0x00f2 (242)
Flags: 0x00
Fragment offset: 24
Time to live: 64
Protocol: UDP (17)
Header checksum: 0xcfx54 [validation disabled]
[Header checksum status: Unverified]
Source: 10.1.1.1
Destination: 129.111.30.27
[Source GeolP: Unknown]
[Destination GeolP: AS20971 University of Texas, United States, San Antonio, TX, 29.507200, -98.574799]
+ 2 IPv4 Fragments (28 bytes): #8(30), #9(4)]
+ [Frame 8, payload: 0-35 (36 bytes)]
+ [Frame 9, payload: 24-27 (4 bytes)]
(Fragment count: 2)
[Reassembled IPv4 length: 28]
[Reassembled IPv4 data: 7cabDee502400000000000000000000000000000000...]
+ User Datagram Protocol, Src Port: 31915, Dst Port: 20197
Source Port: 31915
Destination Port: 20197
Length: 36 ( bogus, payload length 28 )
Checksum: [missing]
Checksum Status: Not present
```

1.3.2.1 Abusing Fragmentation & Detection

You may think that there is no real value in studying older/patched attacks, but older attack mechanics are constantly being updated and reused to discover new vulnerabilities.

This was the case with [CVE-2018-5391](https://portal.msrc.microsoft.com/en-US/security-guidance/advisory/ADV180022) (aka FragmentSmack). This attack resembled the way the original Teardrop attack was executed and affected a plethora of modern Windows (and Linux) targets.